



21. Back Tracking

- ♦ Backtracking is an Algorithmic Technique to find all solutions to a problem.
- ♦ It builds set of all the solutions incrementally. Since a problem would have constraints, the solutions that fail to satisfy constraints, will be removed.
- ♦ It uses recursive calling to find a solution set by building a solution step by step, increasing levels with time.
- ♦ In order to find these solutions, Search Tree named **State-Space Tree** is used.
- ♦ Backtracking algorithm uses the depth-first search(DFS) method.
- ♦ When it starts exploring the solutions, a bounding function(**Condition**) is applied so that the algorithm can check if the so-far built solution satisfies the constraints. If it does, it continues searching. If it doesn't, the branch would be eliminated, and the algorithm goes back to the level before.

21.1. Understand with an Example

- ♦ Find the Permutations of ABC
- ♦ Bounding Function(**Condition**): B should not be after A immediately

Steps:

- 1) According to the backtracking, first, we'll build a **State-Space Tree**.
- 2) We'll find all the possible solutions and check them with the given constraint.
- 3) We'll only keep those solutions that satisfy the given Bounding Function:

All Possible Solutions:

ABC, ACB, BAC, BCA, CBA, CAB

Solutions Passing the Condition:

ACB, BAC, BCA, CBA



21.2. When to Use a Backtracking

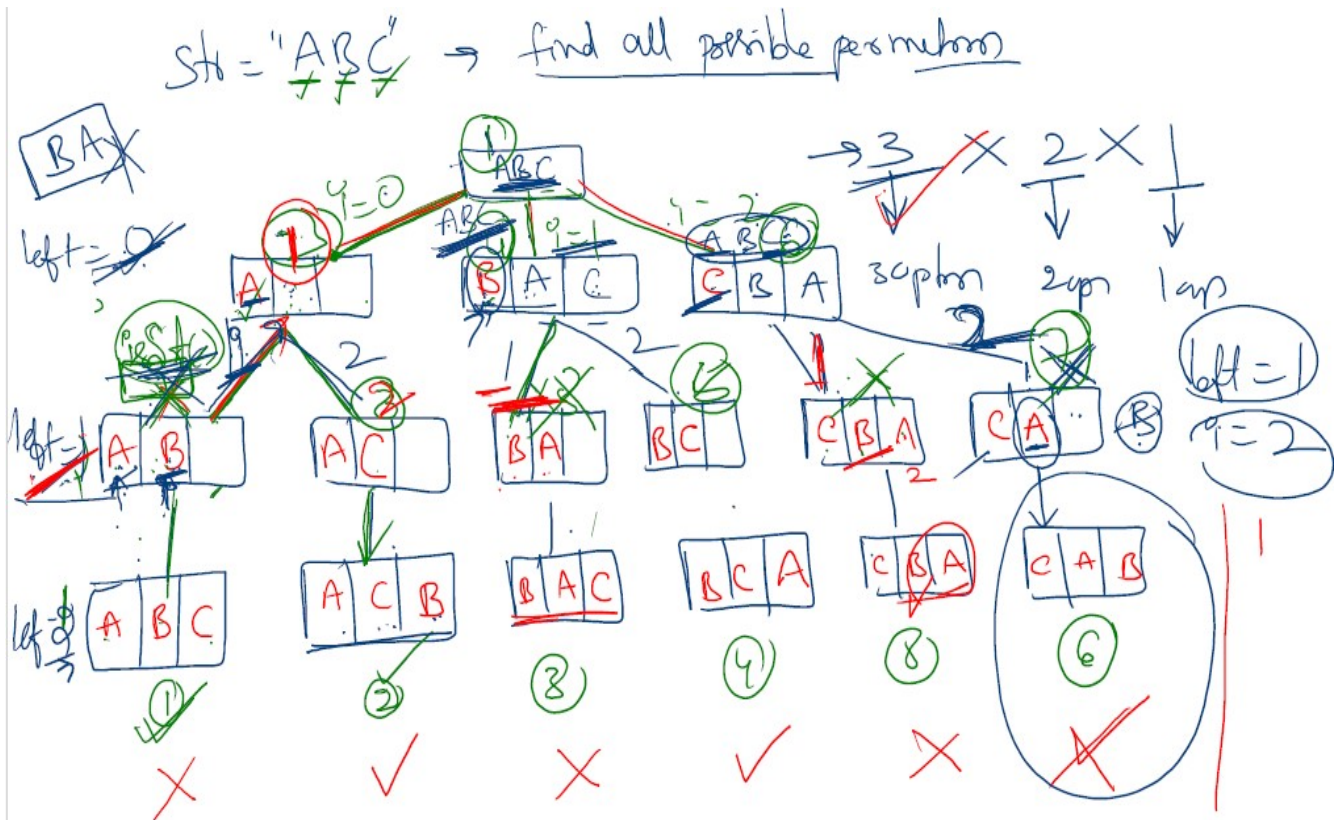
- ♦ Backtracking algorithm is used to find a feasible solution to a decision problem.
- ♦ Backtracking algorithm is used for the enumeration problem in order to find the set of all feasible solutions for the problem.
- ♦ Backtracking algorithms are ideally used when solving problems that involve making a sequence of choices, with each choice leading towards a potential solution.

IMP: Backtracking is not considered an optimized technique to solve a problem. It finds its application when the solution needed for a problem is not time-bounded.

21.3. Applications of Backtracking

- ♦ There are various applications of backtracking algorithms.
- ♦ Some of them are following:
 - 1) Maize solve the problem
 - 2) N- Queens problem
 - 3) Subset Sum problem
 - 4) The Knight's Tour problem
 - 5) Word break problem
 - 6) Word search problem
 - 7) Finding all Hamiltonian Paths present in the graph
 - 8) Longest possible roue in matrix with hurdlesetc

21.4. Problems on BackTracking





Lab1.java

```
package com.jlcindia.backtracking;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Permutations
//Recursive Solution
public class Lab1 {
    static int count = 0;
    static String swap(String str, int i, int left) {
        char ch[] = str.toCharArray();
        char temp = ch[i];
        ch[i] = ch[left];
        ch[left] = temp;
        return new String(ch);
    }
    static void permutation(String str, int left, int right) {
        System.out.println(++count);
        if (left == right) {
            System.out.println(str);
            return;
        }
        for (int i = left; i <= right; i++) {
            str = swap(str, i, left);
            permutation(str, left + 1, right);
            str = swap(str, i, left);
        }
    }
    public static void main(String[] args) {
        String str = "ABC";
        permutation(str, 0, str.length() - 1);
        System.out.println("R-Calls : "+count);
    }
}
```



Lab2.java

```
package com.jlcindia.backtracking;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Permutations - Condition : AB should not appear
//Recursive Solution
public class Lab2 {
    static int count = 0;
    static String swap(String str, int i, int left) {
        char ch[] = str.toCharArray();
        char temp = ch[i];
        ch[i] = ch[left];
        ch[left] = temp;
        return new String(ch);
    }
    static void permutation(String str, int left, int right) {
        System.out.println(++count);
        if (left == right) {
            if (!str.contains("AB"))
                System.out.println(str);
            return;
        }
        for (int i = left; i <= right; i++) {
            str = swap(str, i, left);
            permutation(str, left + 1, right);
            str = swap(str, i, left);
        }
    }
    public static void main(String[] args) {
        String str = "ABCDEFGG";
        permutation(str, 0, str.length() - 1);
        System.out.println("R-Calls : " + count);
    }
}
```



Lab3.java

```
package com.jlcindia.backtracking;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Permutations - Condition : AB should not appear
//BackTracking Solution

public class Lab3 {
    static int count = 0;

    static String swap(String str, int i, int left) {
        char ch[] = str.toCharArray();
        char temp = ch[i];
        ch[i] = ch[left];
        ch[left] = temp;
        return new String(ch);
    }

    static boolean isSafe(String str, int left, int i, int right) {
        if (left != 0 && str.charAt(left - 1) == 'A' && str.charAt(i) == 'B')
            return false;

        if (left != 0 && str.charAt(left - 1) == 'B' && str.charAt(i) == 'A')
            return false;

        if (right == (left + 1) && str.charAt(i) == 'A' && str.charAt(left) == 'B')
            return false;

        if (right == (left + 1) && str.charAt(i) == 'B' && str.charAt(left+1) == 'A')
            return false;

        return true;
    }
}
```



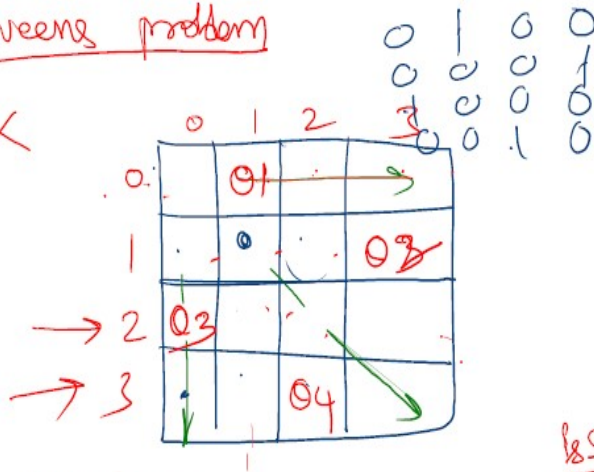
```
static void permutation(String str, int left, int right) {
    System.out.println(++count);
    if (left == right) {
        if (!str.contains("AB"))
            System.out.println(str);
        return;
    }

    for (int i = left; i <= right; i++) {
        if (isSafe(str, left, i, right)) {
            str = swap(str, i, left);
            permutation(str, left + 1, right);
            str = swap(str, i, left);
        }
    }
}

public static void main(String[] args) {
    String str = "ABC";
    permutation(str, 0, str.length() - 1);
    System.out.println("R-Calls : " + count);
}
}
```

N-Queens problem

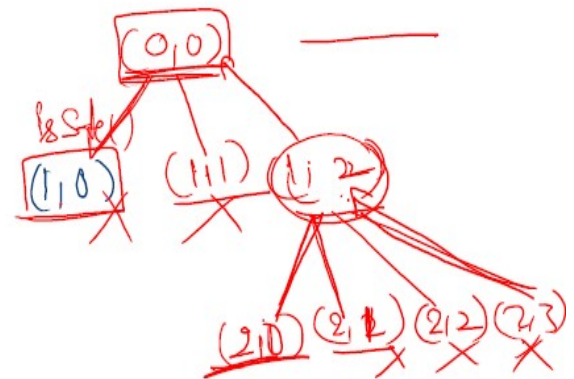
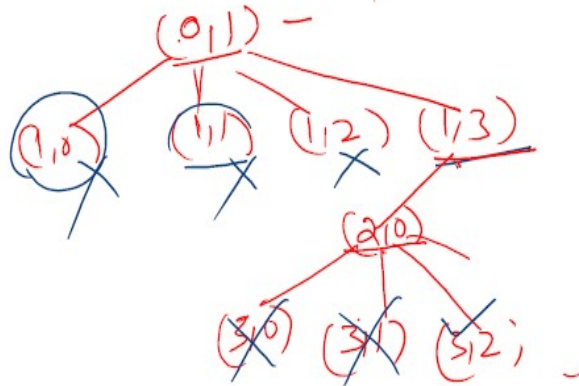
Same Row
Same Col
diagonal



✓ 4x4 chess Board

✓ 4 queens

✓





Lab4.java

```
package com.jlccindia.backtracking;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//N-Queens Problem
//BackTracking Solution
public class Lab4 {

    static void printBoard(int chessBoard[][]) {
        for (int i = 0; i < chessBoard.length; i++) {
            for (int j = 0; j < chessBoard[i].length; j++) {
                System.out.print(chessBoard[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    static boolean isSafe(int chessBoard[][], int row, int col) {
        //Check the Vertical
        for (int i = 0; i < row; i++) {
            if (chessBoard[i][col] == 1) {
                return false;
            }
        }

        //Diagonal Left
        int left = Math.min(row, col);
        for (int i = 1; i <= left; i++) {
            if (chessBoard[row - i][col - i] == 1) {
                return false;
            }
        }
    }
}
```

```
//Diagonal Right
int right = Math.min(row, chessBoard.length-col-1);
for(int i=1;i<=right;i++) {
    if(chessBoard[row-i][col+i]==1) {
        return false;
    }
}

return true;
}

static int nQueens(int chessBoard[][], int row) {

    if (row == chessBoard.length) {
        printBoard(chessBoard);
        System.out.println("-----");
        return 1;
    }

    int count = 0;
    // Place the Queen and Check the Condition
    for (int col = 0; col < chessBoard.length; col++) {
        if (isSafe(chessBoard,row,col)) {
            chessBoard[row][col] = 1;
            count = count + nQueens(chessBoard, row + 1);
            chessBoard[row][col] = 0;
        }
    }
    return count;
}

public static void main(String[] args) {
    int n = 5;
    int chessBoard[][] = new int[n][n];
    int solutionsCount = nQueens(chessBoard, 0);
    System.out.println("Count : " + solutionsCount);
}
}
```